

Spectroscopic Follow-up Orchestration Layer

Design Document for MAGNETS Collaboration Review

Akum Gill

Harvard University

Collaborators: Chris Stubbs, Ashley Villar's group (Yize Dong)

April 2026

Draft for Review

Abstract

This document describes the design and implementation of an orchestration layer for automated spectroscopic follow-up of transients discovered by the RubinAlerts pipeline. The system connects target requests to automated observing plan generation for LLAMAS on Magellan/Baade, providing composite prioritization and multi-program time accounting for the MAGNETS collaboration.

Contents

1	Overview	2
1.1	System Context	2
1.2	MAGNETS Collaboration Context	2
2	Requirements	2
2.1	Functional Requirements	2
2.2	Non-Functional Requirements	3
3	Architecture	3
3.1	Data Flow	3
3.2	Module Responsibilities	4
4	Component Details	4
4.1	Target Normalization	4
4.1.1	Merit-to-Priority Mapping	4
4.2	Composite Priority Scoring	5
4.3	Time Accounting	5
4.4	Greedy Scheduling Algorithm	6
4.5	Exposure Time Estimation	6
5	CLI Interface	6
6	Output Artifacts	7
7	Key Design Decisions	7

8	Planned Features	7
8.1	Google Sheet Integration	7
9	Open Questions for Review	8
10	References	8
10.1	Local Files	8
10.2	Proposal Details (Stubbs 2026B, propid 2835)	8

1 Overview

This infrastructure supports the MAGNETS collaboration (Magellan partner institutions pooling time for Rubin transient follow-up) and implements the “collaborative infrastructure needed to execute this blended program” (Stubbs 2026B proposal).

1.1 System Context

The orchestration layer sits between:

- **Upstream:** RubinAlerts pipeline candidates, manual target requests
- **Downstream:** LLAMAS observing plans, time accounting records

1.2 MAGNETS Collaboration Context

From the Stubbs 2026B proposal (propid 2835):

“MAGNETS, a collaboration between several Magellan partner institutions, is a brand new effort motivated by the start of LSST alerts in March of this year... Our collaboration’s plan is to pool our awarded observing time and develop an internal queue schedule to address a wide range of science goals.”

Key requirements from proposal:

- Queue-based observational strategy
- Time apportioned in proportion to TAC awards
- Multiple science cases: SNe Ia cosmology (Stubbs), core collapse (Villar), exotic transients
- Student projects declared and protected

Instrument: LLAMAS Integral Field Spectrograph (Magellan/Baade)

2 Requirements

2.1 Functional Requirements

ID	Requirement	Priority
F1	Ingest target requests from CSV and RubinAlerts	Must
F2	Normalize heterogeneous inputs to common schema	Must
F3	Score/rank targets by composite priority	Must
F4	Track time allocations per program/PI	Must
F5	Generate observing plans for LLAMAS	Must
F6	Handle backup target lists	Should
F7	Integrate with RubinAlerts merit-scored candidates	Should
F8	Support white dwarf spectrophotometric standards	Should
F9	Moon phase-aware scheduling (dark/grey/bright)	Should
F10	Google Sheet integration for manual requests	Planned

Table 1: Functional requirements

2.2 Non-Functional Requirements

ID	Requirement
NF1	Run nightly without manual intervention
NF2	Produce human-readable audit trail
NF3	Graceful degradation if external APIs fail
NF4	Compatible with existing SkyPortal data flow

Table 2: Non-functional requirements

3 Architecture

3.1 Data Flow

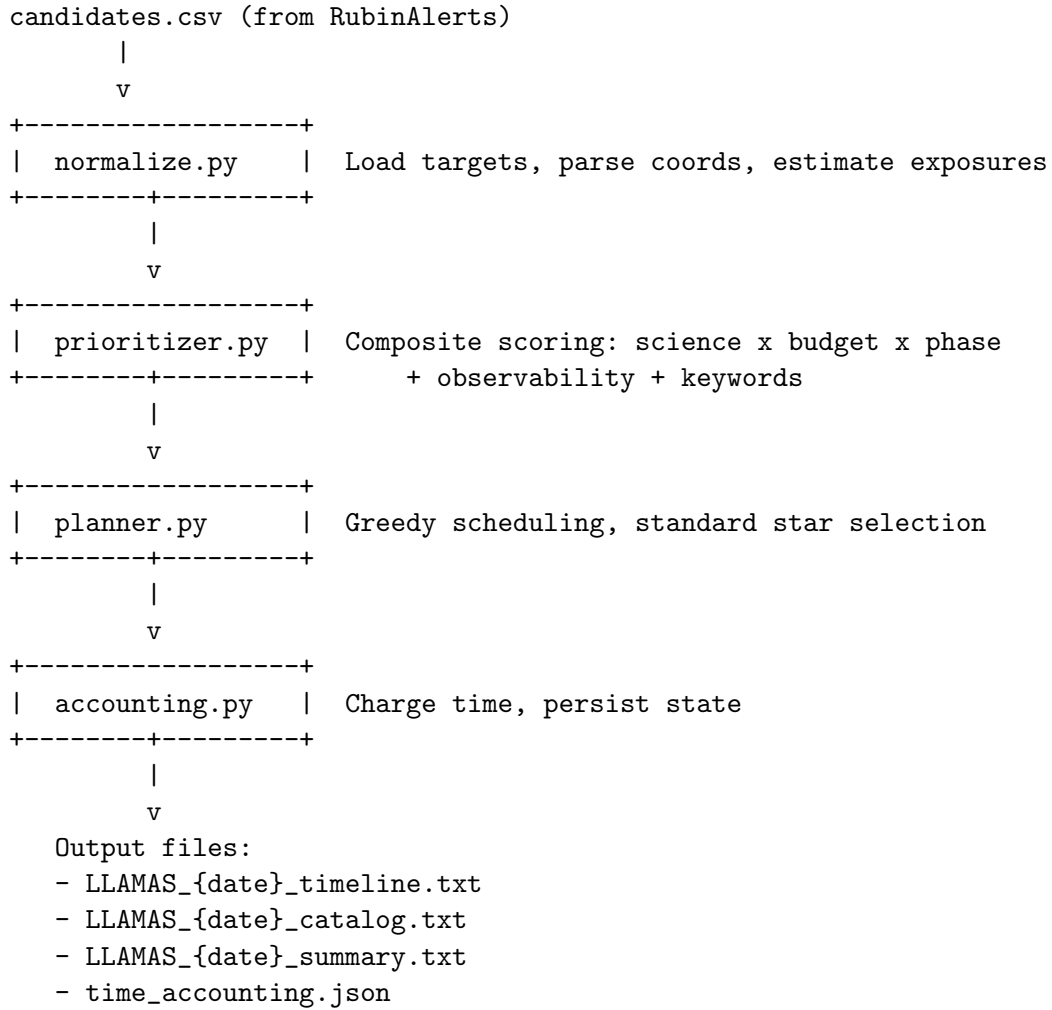


Figure 1: LLAMAS orchestrator data flow

3.2 Module Responsibilities

Module	Purpose
<code>config.py</code>	LLAMASConfig: LCO site (-29.01° , -70.69° , 2380m), 1-min IFU overhead, airmass limit 1.6, exposure table from proposal
<code>models.py</code>	Target, ScheduledEntry, ProgramAllocation, ObsPlan dataclasses
<code>normalize.py</code>	Load CSV/RubinAlerts targets, parse coordinates, estimate exposures
<code>planner.py</code>	Twilight calculation, observability windows, greedy scheduling, standard star selection
<code>prioritizer.py</code>	Composite scoring: science $\times$ budget $\times$ phase $\times$ observability + keywords
<code>accounting.py</code>	TimeAccountant: per-program budgets (D/G/B), charge/reconcile, JSON persistence
<code>output.py</code>	Timeline, Magellan TCS catalog, human-readable summary writers
<code>reporting.py</code>	Nightly time reports, season progress with burn rate projections

Table 3: Module responsibilities

4 Component Details

4.1 Target Normalization

Converts heterogeneous inputs to canonical schema:

```
@dataclass
class Target:
    name: str
    ra_deg: float
    dec_deg: float
    coord: SkyCoord # Computed from ra_deg, dec_deg

    mag: float
    mag_filter: str # 'r', 'g', 'i', etc.
    redshift: float # For exposure estimation

    priority: int # 1 (highest) to 4 (lowest)
    program: str # For time accounting

    exposure_minutes: float
    n_exposures: int

    merit_score: float # From RubinAlerts (optional)
    phase_weight: float # w_time from alert pipeline
```

4.1.1 Merit-to-Priority Mapping

When loading from RubinAlerts candidates, merit scores are converted to priorities via quartile mapping:

- Top 25% $\rightarrow$ P1

- 25–50% → P2
- 50–75% → P3
- Bottom 25% → P4

4.2 Composite Priority Scoring

The prioritizer computes a composite score combining multiple factors:

$$\text{score} = (\text{science} \times 100 \times \text{budget} \times \text{phase}) + (\text{observability} \times 20) + (\text{keywords} \times 50) \quad (1)$$

Components:

- **science_weight:** From explicit priority (P1=1.0, P2=0.7, P3=0.4, P4=0.2)
- **budget_factor:** 1.0 if >5h remaining, 0.5 if 0–5h, 0.1 if exhausted
- **phase_weight:** w_{time} from alert pipeline merit function
- **observability:** $\text{window_hours} / \text{night_hours}$ (0–1)
- **keyword_adjustment:** “HIGH PRIORITY” +0.3, “backup” –0.3, etc.

The phase weight from the alert pipeline is:

$$w_{\text{time}} = \exp\left(-\frac{\Delta t^2}{2\tau^2}\right), \quad \tau = 10 \text{ days} \quad (2)$$

This automatically boosts targets near peak brightness.

4.3 Time Accounting

Multi-program budgets tracked per moon phase:

```
# allocations.yaml
semester: "2026B"
default_program: "MAGNETS-Stubbs"
programs:
- program: "MAGNETS-Stubbs"
  pi: "Stubbs"
  allocated_hours:
    dark: 5.0
    grey: 20.0
    bright: 5.0
- program: "MAGNETS-Villar"
  pi: "Villar"
  allocated_hours:
    dark: 4.0
    grey: 8.0
    bright: 4.0
```

Charge-on-schedule: Time is debited when the plan is generated. Post-night reconciliation adjusts for weather losses or target changes. Persistent JSON state maintains a full charge log for audit.

4.4 Greedy Scheduling Algorithm

The scheduler uses a greedy algorithm adapted from Alex’s LDSS3 script:

$$\text{schedule_score} = \text{composite_priority} - \text{airmass} \times 10 \quad (3)$$

At each time step, the scheduler selects the observable target with the highest score. Falls back to $(5 - \text{priority}) \times 100 - \text{airmass} \times 10$ when no prioritizer is active (backward compatibility with `plan` command).

4.5 Exposure Time Estimation

Three-tier cascade from Stubbs 2026B proposal Table 1:

Strategy	Input Required	Example	Fallback
Redshift table	z known	$z = 0.25 \rightarrow 95$ min (grey)	—
Magnitude scaling	mag known	mag 20 $\rightarrow$ 45 min, $2.5\times/\text{mag}$	If no z
Default	neither	45 min	If no mag

Table 4: Exposure estimation cascade

Moon phase scheduling (from proposal Table 1):

Redshift	Moon	Peak r	Exposure	Binning
0.10–0.20	Bright	19.3	35 min	$2\times$
0.20–0.30	Grey	20.6	95 min	$2\times$
0.30–0.35	Grey/Dark	21.3	45–160 min	$4\times$
0.35–0.40	Dark	21.9	180 min	$4\times$

Table 5: Exposure times by redshift and moon phase (Stubbs 2026B proposal)

5 CLI Interface

```
# Basic plan from CSV
python -m orchestrator plan --date 2026-10-15 \
    --targets targets.csv --moon grey --output-dir output/

# Full nightly run with time accounting
python -m orchestrator run-nightly --date 2026-10-15 \
    --candidates candidates.csv \
    --allocations allocations.yaml \
    --moon grey --output-dir output/

# Post-night reconciliation
python -m orchestrator reconcile \
    --allocations allocations.yaml \
    --program MAGNETS-Stubbs \
    --actual-hours 3.5 --moon grey --date 2026-10-15
```

```
# Backward compatibility (no subcommand = plan)
python -m orchestrator --date 2026-10-15 --targets targets.csv --moon grey
```

6 Output Artifacts

File	Contents
LLAMAS_{date}_timeline.txt	UT observing sequence with times
LLAMAS_{date}_catalog.txt	Magellan TCS 16-field catalog format
LLAMAS_{date}_summary.txt	Human-readable plan with budget tracking
time_accounting.json	Persistent time charges and audit log
season_report.txt	Progress summary with burn rate projections

Table 6: Output artifacts

7 Key Design Decisions

1. **LLAMAS only** — Orchestrator is for LLAMAS on Magellan/Baade exclusively. LDSS3 materials are reference only.
2. **1-minute IFU overhead** — LLAMAS’s integral field unit eliminates slit alignment, giving $\sim 10\times$ better overhead than slit spectrographs.
3. **Composite priority with phase awareness** — Targets near peak brightness are automatically boosted via the phase weight from the alert pipeline.
4. **Charge-on-schedule with reconciliation** — Simple, prevents over-scheduling. Post-night reconciliation handles weather and target changes.
5. **Greedy scheduling** — Fast ($O(N \log N)$), near-optimal for single-night scheduling.
6. **Quartile-based priority mapping** — Automatic P1–P4 assignment from merit scores ensures fair distribution across priority levels.

8 Planned Features

8.1 Google Sheet Integration

Allow PIs to manually enqueue targets not sourced from alerts:

```
import gspread
from google.oauth2.service_account import Credentials

SCOPES = ['https://www.googleapis.com/auth/spreadsheets.readonly']

def fetch_sheet(sheet_id: str, range_name: str) -> list[dict]:
    creds = Credentials.from_service_account_file(
        'credentials.json', scopes=SCOPES)
    client = gspread.authorize(creds)
    sheet = client.open_by_key(sheet_id)
```

```
worksheet = sheet.worksheet(range_name)
return worksheet.get_all_records()
```

Use case: PIs want to follow up interesting targets discovered through other channels (other surveys, ToO triggers, collaboration requests).

9 Open Questions for Review

1. **Time accounting granularity:** Currently charge-on-schedule with post-night reconciliation. Is this sufficient, or do we need more sophisticated tracking?
2. **Priority conflicts:** How should we handle when multiple programs request the same target? Currently first-come-first-served based on plan generation order.
3. **Backup target policy:** Should backups count against time budget when scheduled? Currently they do.
4. **Multi-night planning:** Should we look ahead to optimize across nights? Currently single-night greedy.
5. **Keyword vocabulary:** Current keywords include “HIGH PRIORITY”, “classification needed”, “rising”, “near peak”, “backup”, “low prio”, “if time”. Are these sufficient?
6. **Priority weight tuning:** Current weights (science $\times$ 100, observability $\times$ 20, keywords $\times$ 50) were chosen heuristically. Should these be configurable or tuned?

10 References

10.1 Local Files

- Alex’s LDSS3 script: `ref/LDSS_ObsPlan_Generator/generate_obsplan.py`
- Stubbs 2026B proposal: `ref/cstubbs2026B (2) (1).pdf`
- RubinAlerts pipeline: `run_tonight.py`, `core/magellan_planning.py`

10.2 Proposal Details (Stubbs 2026B, propid 2835)

- Semester: 2026B (Jul 7 – Jan 16)
- Instrument: LLAMAS Integral Field Spectrograph
- Telescope: Magellan Baade
- Nights: 0.5D + 2.0G + 0.5B (30 hours total)
- Targets: 29 (SNe Ia in DDFs + WD standards)
- Magnitude range: $18.0 \leq r \leq 21.5$
- Redshift range: $0.1 < z < 0.4$